

Übungsblatt 2

Scheduling

Abgabe bis Di., 12. Mai, 14 Uhr.

Für das erfolgreiche Absolvieren der Übungen müssen Sie sich einmalig in FlexNow zum Übungsmodul **B.Inf.1102.Ue: Grundlagen der Praktischen Informatik - Übung** anmelden.

Melden Sie sich **rechtzeitig** in **FlexNow** an. Eine nachträgliche Anmeldung nach Ende der Anmeldefrist ist **nicht** möglich.

Rechnerübung: Hilfe zu den Übungszetteln erhalten Sie in den Rechnerübungen im CIP-Pool -1.110 (IFI). Eine Übersicht über die Zeiten der Rechnerübungen finden Sie im Informations-Reiter der Stud.IP-Veranstaltung. Eine Terminreservierung für Fragen während der Rechnerübung ist nicht notwendig.

Achtung: Testate der praktischen Aufgaben haben Vorrang gegenüber der Rechnerübung, es kann also vereinzelt zu Wartezeiten kommen.

Für die Prüfungszulassung benötigen Sie auf 8 verschiedenen Übungsblättern sowohl im theoretischen als auch im praktischen Teil jeweils 40%, also jeweils 40 Punkte.

Theoretischer Teil (100 Punkte)

Hinweise zur Abgabe der Lösungen:

Die Lösungen der Aufgaben werden im **PDF-Format** in der Stud.IP-Veranstaltung der Vorlesung über das Vips-Modul hochgeladen. Sie können Ihre Bearbeitungen mit $\text{\LaTeX}$ formatieren, es ist aber auch der Upload durch andere Formatierungsmöglichkeiten zulässig.

Sollten Sie $\text{\LaTeX}$ nutzen wollen, gibt es hierfür Hinweise in Form eines Videos in Stud.IP und Sie können unser $\text{\LaTeX}$ -Template für Ihre Abgaben nutzen.

Die Aufgaben können alleine oder zu zweit (als Gruppe) bearbeitet und abgegeben werden. Wenn Sie in einer Gruppe abgeben möchten, achten Sie darauf, dass Sie sich vor der Abgabe im Vips-Modul der Vorlesung in einer Gruppe zusammenschließen.

Vorbereitung - Rechenintensive Prozesse und Scheduling

Allgemeine Begriffe

- Ein Prozess wird **aktiviert**, wenn er Prozessorzeit zugeteilt bekommt, d.h. der Zustand des Prozesses wechselt von 'bereit' zu 'rechnend'.
- Die **Ankunftszeit** eines Prozesses ist der Zeitpunkt, ab dem der Prozess vom Scheduling berücksichtigt wird. Der Prozess wird erzeugt, ist rechenbereit und wird der Menge der Prozesse mit Zustand 'bereit' hinzugefügt. Ist dieser Prozess der einzige in der Menge der Prozesse mit Zustand 'bereit', wird der Prozess sofort aktiviert.

Beispiel. Kommt der Prozess P zum Zeitpunkt t an, ist die Bereitliste leer und kein Prozess belegt den Prozessor, dann wird der Prozess P zum Zeitpunkt t aktiviert.

- Die **Endzeit** ist der Zeitpunkt, an dem ein Prozess vollständig abgelaufen ist.
- Die **Wartezeit** eines Prozesses ist die Zeit zwischen Ankunfts- und Endzeitpunkt, in der der Prozess nicht rechnet.

Nicht-unterbrechendes Scheduling

- Ein Prozess läuft nach der Aktivierung vollständig ab ohne zu blockieren.
- Kommt ein neuer Prozess an, wird er in die Menge der bereiten Prozesse eingeordnet.
- Beendet sich der aktuell rechnende Prozess, muss ein Prozess aus der Menge der bereiten Prozesse ausgesucht werden, der aktiviert wird. Ist die Menge leer, läuft der Prozessor (quasi im Leerlauf) weiter, bis wieder ein Prozess in der Menge der bereiten Prozesse verfügbar ist.

Unterbrechendes Scheduling

- Der aktuell rechnende Prozess läuft ohne zu blockieren solange bis er unterbrochen wird oder vollständig abgelaufen ist, d.h. sich beendet.
- Kommt ein neuer Prozess an, wird er in die Menge der bereiten Prozesse eingeordnet. Weiterhin wird der aktuell rechnende Prozess unterbrochen und ebenfalls in die Menge der bereiten Prozesse eingeordnet, dabei wird die noch verbleibende Rechenzeit dem Prozess als Rechenzeit zugeordnet.
- Kommt ein neuer Prozess an oder beendet sich der aktuell rechnende Prozess, muss ein Prozess aus der Menge der bereiten Prozesse ausgesucht werden, der aktiviert wird. Ist die Menge leer, läuft der Prozessor (quasi im Leerlauf) weiter, bis wieder ein Prozess in der Menge der bereiten Prozesse verfügbar ist.

Aufgabe 1 (50 Punkte)

Hinweis: Lesen Sie zuerst die Informationen zu Scheduling auf der **Vorseite** durch, bevor Sie die Aufgabe bearbeiten.

Auf der Suche nach einem guten Scheduling-Verfahren für Großrechner, auf denen hauptsächlich rechenintensive Prozesse ablaufen, wird folgendes Beispiel betrachtet.

Prozesse	P_1	P_2	P_3	P_4	P_5	P_6
Ankunftszeit	0	13000	14000	39000	41000	47000
Rechenzeit	15000	20000	5000	50000	2000	10000

1. Lassen Sie die Prozesse $P_1, \dots, P_6$ mit nicht-unterbrechendem und unterbrechendem Scheduling ablaufen. Stellen Sie jeweils, wenn ein Prozess aktiviert wird, den rechnenden Prozess und die Menge der bereiten Prozesse dar.

Wählen Sie jeweils den zu aktivierenden Prozess, sodass die Summe der Wartezeit für die Prozesse in der Menge der bereiten Prozesse möglichst klein wird.

Bestimmen Sie die durchschnittliche Wartezeit pro Prozess.
(20 Punkte)

2. Leiten Sie aus dem Beispiel für nicht-unterbrechendes und unterbrechendes Scheduling ein allgemeines Kriterium für die Wahl des jeweils zu aktivierenden Prozesses ab, sodass die durchschnittliche Wartezeit möglichst klein wird.
 - Welche Strategie sollte beim nicht-unterbrechenden Scheduling angewendet werden? (10 Punkte)
 - Welche Strategie sollte beim unterbrechenden Scheduling angewendet werden? (10 Punkte)
3. Weist man den einzelnen Prozessen Prioritäten zu, kann beim Scheduling anhand dieser Prioritäten entschieden werden, welcher Prozess als nächstes gewählt wird, das heißt, ein Prozess mit höherer Priorität wird bevorzugt. Dies kann sowohl in einem unterbrechenden als auch in einem nicht-unterbrechenden Szenario betrachtet werden.

Geben Sie Vor- und Nachteile von Scheduling mit Prioritäten an. Gibt es Unterschiede zwischen einem nicht-unterbrechenden und unterbrechenden Szenario? (10 Punkte)

Aufgabe 2 (50 Punkte)

Betrachten Sie folgendes Beispiel, welches mit Round-Robin mit Zeitscheibenlänge 10 ausgeführt wird.

Prozesse	P_1	P_2	P_3	P_4	P_5	P_6
Ankunftszeit	0	5	7	10	15	20
Rechenzeit	6	5	10	2	1	8

1. Was beobachten Sie im obigen Beispiel bei der gegebenen Zeitscheibenlänge von 10? (10 Punkte)
2. Beschreiben Sie, was passiert, wenn beim Round-Robin-Verfahren die Zeitscheibenlänge zu klein oder zu groß gewählt wird. (15 Punkte)
3. Welches Ihnen aus der Vorlesung bekannte Scheduling-Verfahren würde in dem oben dargestellten Beispiel eine bessere durchschnittliche Wartezeit erreichen? (15 Punkte)
4. Beschreiben die Funktionsweise eines Scheduling-Algorithmus, der die Prozesse in einzelne Gruppen aufteilt. Gehen Sie dabei auf folgende Punkte ein.
 - (a) In welche Gruppen könnten Prozesse aufgeteilt werden? (3 Punkte)
 - (b) Welchen Gruppen sollte eine höhere Priorität zugewiesen werden? (3 Punkte)
 - (c) Wie werden die einzelnen Gruppen verwaltet? (4 Punkte)

Praktischer Teil (100 Punkte)

Testate von Di., 12. Mai, 18 Uhr bis Di., 19. Mai, 14 Uhr.

Im Gegensatz zum theoretischen Teil müssen Sie im praktischen Teil ausführbare Programme schreiben. Die Abgabe erfolgt somit nicht im PDF-Format, sondern in einem von der jeweiligen Programmiersprache abhängigen **geeigneten Dateiformat**. Erfordert die Aufgabenstellung, ein Programm in Haskell zu produzieren, geben Sie bitte Ihre Lösungen in einer oder mehreren **.hs**-Dateien ab; die Abgabe von Python-Programmen kann mittels **.py**-Dateien oder in Jupyter-Notebooks erfolgen.

Die Bewertung des praktischen Teils erfolgt in einem **Testat**. Hierbei präsentieren Sie einem Tutor Ihre Lösungen und erhalten Punkte auf die erfolgreiche Erklärung. Die Testate finden zeitgleich mit den Rechnerübungen im CIP-Pool -1.110 (IfI) statt. Die Zeiten der Rechnerübungen/Testate können Sie im Informationen-Reiter der Stud.IP-Veranstaltung nachsehen.

1. Laden Sie alle Dateien, die Sie während des Testats benötigen, über das Vips-Modul der Stud.IP-Veranstaltung in der Aufgabe *Übung 02 - Testat* hoch. Nutzen Sie hierfür geeignete Dateiformate: Geben Sie beispielsweise Haskell-Aufgaben in **.hs**-Dateien ab. Die **rechtzeitige** Abgabe in Vips ist verpflichtend, um die Chancengleichheit bei verschiedenen Testatzeiten zu gewährleisten.
2. Um die praktische Übung testieren zu lassen, müssen Sie einen Termin über die Stud.IP-Terminvergabe für ein Testat **reservieren**. Ein Testat ohne Termin ist **nicht möglich**.
3. Die Testate starten immer dienstags um 18 Uhr, nach Abgabe des jeweiligen Übungszettels, und enden vor der Abgabefrist des nächsten Übungszettels (dienstags, 14 Uhr).
4. Die Testate können allein oder in Zweiergruppen absolviert werden. Dabei sind die Gruppen identisch zu denen, die auch die theoretischen Aufgaben zusammen bearbeiten. In diesem Fall reserviert nur ein Gruppenmitglied einen Termin.
5. Bringen Sie bzw. jedes Mitglied Ihrer Gruppe zu jedem Testat den Kontrollzettel **Praktische Übung - Testate** (siehe Übungsblatt 1) mit. Ein Einspruch zu nicht oder falsch eingetragenen Punkten erfordert die Vorlage des Kontrollzettels.

Es stehen jede Woche nur eine begrenzte Anzahl an Testatterminen zur Verfügung. Nutzen Sie daher bitte die gesamte Breite an Terminen. Eine nachträgliche Testierung ist nur in begründeten Ausnahmefällen möglich.

Aufgabe 1 (100 Punkte)

1. Programmieren Sie eine Funktion

```
1 foldList :: (Double -> Double -> Double) -> [Double] -> Double
```

die alle Elemente einer Liste mit Hilfe der/des übergebenen Funktion/Operators (`Double -> Double -> Double`) zusammenfasst, entsprechend nachfolgender rekursiver Definition mit beliebiger Funktion $f : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ und Folge $x_n, \dots, x_0$, wobei $x_i \in \mathbb{R}$.

$$\text{foldList}_f(x_n, \dots, x_0) = \begin{cases} x_0 & \text{für } n = 0 \\ f(x_n, \text{foldList}_f(x_{n-1}, \dots, x_0)) & \text{sonst} \end{cases}$$

Benutzen Sie nicht die Funktion `fold` aus `prelude`. Die Funktion sollte wie im folgenden Beispiel funktionieren.

```
1 > foldList (+) [1.0..100.0]
2 5050.0
```

(20 Punkte)

Hinweis.

- `[1.0..10.0]` ist die Liste der ganzzahligen Gleitkommazahlen von 1.0 bis 10.0.
- Sie können davon ausgehen, dass die Funktion nur mit einer nicht-leeren Liste aufgerufen wird.

2. Programmieren Sie eine Funktion

```
1 mapList :: (Int -> Int) -> [Int] -> [Int]
```

die als Funktionswert eine Liste von ganzen Zahlen `[Int]` liefert, welche durch Anwendung einer übergebenen Funktion (`Int -> Int`) auf jedes Element einer übergebenen Liste von ganzen Zahlen `[Int]` entsteht.

Benutzen Sie nicht die Funktion `map` aus `prelude`. Die Funktion sollte wie im folgenden Beispiel funktionieren.

```
1 > mapList square [1..10]
2 [1,4,9,16,25,36,49,64,81,100]
3 > mapList (quadratic (1,2,4)) [1..10]
4 [7,12,19,28,39,52,67,84,103,124]
```

(20 Punkte)

Hinweis.

- `[1..10]` ist die Liste der ganzen Zahlen von 1 bis 10.

3. Programmieren Sie eine Funktion

```
1 containsList :: [Int] -> Int -> Bool
```

die überprüft, ob ein Wert `Int` in einer Liste von Werten `[Int]` enthalten ist und einen entsprechenden Wahrheitswert `Bool` zurückliefert.

Benutzen Sie nicht die Funktion `elem` aus `prelude`, sondern **verwenden Sie Rekursion**. Die Funktion sollte wie im folgenden Beispiel funktionieren.

```
1 > containsList [-100..100] (-12)
2 True
```

(30 Punkte)

4. Mit dem Kommando

```
1 > import Data.Char (toLower)
```

können Sie aus dem Modul `Data.Char` die Funktion `toLower :: Char -> Char` importieren, die sich auf alle `Char` anwenden lässt und für Großbuchstaben den zugehörigen Kleinbuchstaben und sonst den `Char` unverändert zurückliefert.

```
1 > import Data.Char (toLower)
2 > toLower 'S'
3 s
4 > toLower 's'
5 s
6 > toLower '#'
7 #
```

Programmieren Sie eine Funktion

```
1 countList :: [Char] -> Char -> Int
```

die zählt, wie oft in der übergebenen Liste `[Char]` der übergebene `Char` enthalten ist und das Ergebnis zurückliefert. Dabei soll nicht zwischen Groß- und Kleinschreibung unterschieden werden (siehe `toLower`).

(30 Punkte)

Hinweis. `[Char]` und `String` haben dieselbe Bedeutung.